Flux de Données et Gestion de l'État Global avec Redux

Objectifs

1. Identifier les limites du passage de `props` en profondeur (*Prop Drilling*) dans une application complexe.
2. Comprendre l'architecture Flux et le cycle de vie unidirectionnel d'un état global.
3. Maîtriser les concepts fondamentaux de Redux : Store, State, Actions, Reducers et Selectors.
4. Mettre en place une gestion d'état moderne et concise avec Redux Toolkit (`configureStore`, `createSlice`).
5. Connecter les composants React au Store global à l'aide des Hooks `useSelector` et `useDispatch`.

1. La Problématique du Partage d'État à Grande Échelle

1.1 Le Piège du Prop Drilling (Forage de Propriétés)

Dans les premiers modules, nous avons vu que pour partager un état entre deux composants, il faut faire remonter cet état (*Lifting State Up*) jusqu'à leur ancêtre commun le plus proche, puis le faire redescendre via les `props`.

Lorsque l'arborescence de l'application s'agrandit (plusieurs dizaines de composants répartis sur plusieurs niveaux de profondeur), cette approche atteint ses limites :

- Des composants intermédiaires (comme `Layout`, `Sidebar` ou `DashboardGrid`) se retrouvent obligés de recevoir et de transmettre des `props` dont ils n'ont absolument pas besoin eux-mêmes, uniquement pour les acheminer vers un composant enfant situé tout en bas de l'arbre.
- C'est ce qu'on appelle le **Prop Drilling**. Il alourdit le code, couple fortement les composants entre eux et provoque des re-rendus en cascade inutiles à chaque modification.

1.2 État Local vs État Global

Toutes les données n'ont pas vocation à devenir globales. Il est essentiel de distinguer deux catégories d'état :

Type d'état	Définition et Cas d'Usage	Outil Recommandé
État Local (UI State)	Donnée propre à un seul composant ou une petite section (ouverture d'une modale, saisie d'un champ texte, onglet actif).	`useState`
État Global (App State)	Donnée partagée par de nombreuses vues éloignées dans l'application (utilisateur connecté, préférences, liste des stations et alertes temps réel).	**Redux (Redux Toolkit)** ou Context API

2. De l'Architecture Flux à Redux

2.1 Le Modèle Architectural Flux

Créé par Facebook en complément de React, **Flux** n'est pas une bibliothèque mais un patron d'architecture (*Design Pattern*) imposant un **flux de données strictement unidirectionnel** au niveau global de l'application.

En 2015, Dan Abramov et Andrew Clark créent **Redux**, une implémentation épurée et prévisible de Flux inspirée de l'architecture du langage fonctionnel Elm.

2.2 Les Trois Principes Fondamentaux de Redux

1. **Source unique de vérité (Single Source of Truth) :** Tout l'état global de votre application est stocké dans un arbre d'objets unique au sein d'un seul **Store**.
2. **L'état est en lecture seule (State is Read-Only) :** Les composants ne peuvent jamais modifier l'état global directement. La seule façon de demander une modification consiste à émettre (**dispatch**) un objet décrivant ce qui s'est passé : une **Action**.
3. **Les changements sont effectués par des fonctions pures (Reducers) :** Pour définir comment l'arbre d'état est transformé par les actions, vous écrivez des fonctions pures appelées **Reducers**. Un Reducer prend l'état précédent et une action, et retourne le nouvel état de manière immuable.

2.3 Le Cycle de Vie d'une Mise à Jour dans Redux

Le cycle complet suit toujours le même chemin circulaire en 4 étapes :

1. **Interaction Vue :** L'opérateur clique sur un bouton `"Couper les capteurs"` dans l'interface React.
2. **Dispatch d'une Action :** Le composant expédie une action normalisée au Store via `dispatch({ type: 'stations/shutdownAll' })`.
3. **Exécution du Reducer :** Le Store transmet l'état actuel et l'action au Reducer. Celui-ci calcule et retourne une nouvelle copie de l'état mis à jour.
4. **Notification et Re-rendu :** Le Store enregistre le nouvel état et notifie uniquement les composants React abonnés à cette donnée précise pour qu'ils se mettent à jour.

3. Redux Moderne avec Redux Toolkit (RTK)

Historiquement, le Redux "classique" (Legacy Redux) était réputé pour sa verbosité extrême : il fallait créer manuellement des constantes de types d'actions, des créateurs d'actions (*Action Creators*), et gérer manuellement des copies immuables complexes avec l'opérateur `...` dans des `switch / case`.

Aujourd'hui, l'équipe officielle de Redux impose l'utilisation de **Redux Toolkit (`@reduxjs/toolkit`)** comme standard industriel unique :

- Il élimine 80 % du code répétitif (*boilerplate*).
- Il regroupe les actions et les reducers d'une même fonctionnalité dans un concept unique appelé **Slice** (tranche d'état).
- Il intègre nativement la bibliothèque **Immer**, qui permet d'écrire une syntaxe d'apparence mutatrice (`state.items.push(...)`) que Redux Toolkit convertit automatiquement en mise à jour 100 % immuable sous le capot.
- Il configure automatiquement l'extension de débogage **Redux DevTools**.

Installation des paquets

Dans un projet React, Redux nécessite deux bibliothèques complémentaires :

- `@reduxjs/toolkit` : Le moteur logique de gestion du Store et des Slices.
- `react-redux` : Le pont officiel permettant de relier les composants React au Store Redux.

Shell

```
npm install @reduxjs/toolkit react-redux
```

4. Mise en Place Pas à Pas d'un Store Redux Toolkit

4.1 Étape 1 : Créer une Tranche d'État (Slice) avec `createSlice`

Une "tranche" (*Slice*) représente une brique métier de votre état global (par exemple `stationsSlice` pour gérer le parc de stations météo, et `alertsSlice` pour gérer les alertes).

La fonction `createSlice` génère automatiquement les créateurs d'actions et le reducer correspondant :

JavaScript

```
// src/store/stationsSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  list: [
    { id: 1, name: 'Toulouse-Blagnac', status: 'online', altitude: 152 },
    { id: 2, name: 'Pic du Midi', status: 'offline', altitude: 2877 }
  ],
  filterStatus: 'all' // 'all', 'online', 'offline'
};

const stationsSlice = createSlice({
  name: 'stations',
  initialState,
  reducers: {
    // 1. Ajouter une station (payload contient le nouvel objet station)
    addStation: (state, action) => {
      // Grâce à Immer intégré dans RTK, ce .push() est converti en opération
immuable !
      state.list.push(action.payload);
    },

    // 2. Basculer le statut d'une station (payload contient l'ID de la station)
    toggleStationStatus: (state, action) => {
      const station = state.list.find(st => st.id === action.payload);
      if (station) {
        station.status = station.status === 'online' ? 'offline' : 'online';
      }
    },
```

```
    // 3. Modifier le filtre actif (payload contient la chaîne du filtre)
    setFilterStatus: (state, action) => {
      state.filterStatus = action.payload;
    }
  }
});

// Export des créateurs d'actions pour les composants
export const { addStation, toggleStationStatus, setFilterStatus } =
stationsSlice.actions;

// Export du reducer pour l'assembler dans le Store
export default stationsSlice.reducer;
```

4.2 Anatomie d'un Objet `action` et de `payload`

Lorsqu'un composant appelle `addStation({ id: 3, name: 'Bordeaux' })`, Redux Toolkit génère automatiquement un objet JavaScript standard structuré ainsi :

JavaScript
```
{
  type: 'stations/addStation', // Préfixe du slice + nom de la fonction reducer
  payload: { id: 3, name: 'Bordeaux' } // La donnée utile transportée par l'action
}
```

4.3 Étape 2 : Assembler le Store Global avec `configureStore`

Le fichier `src/store/store.js` centralise toutes les tranches d'état (*Slices*) de l'application au sein d'un Store unique.

JavaScript
```
// src/store/store.js
import { configureStore } from '@reduxjs/toolkit';
import stationsReducer from './stationsSlice';

export const store = configureStore({
  reducer: {
    stations: stationsReducer
    // Vous pourrez ajouter d'autres tranches ici :
    // alerts: alertsReducer,
```

```
    // auth: authReducer
  }
});
```

4.4 Étape 3 : Injecter le Store dans l'Application avec `<Provider>`

Pour que tous les composants de l'application puissent accéder au Store Redux, il faut envelopper le composant racine `<App />` dans le composant `<Provider>` fourni par `react-redux` au niveau de `src/main.jsx`.

JavaScript
```
// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux';
import { store } from './store/store.js';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </StrictMode>
);
```

5. Consommer et Modifier l'État dans les Composants React

Une fois le `<Provider>` configuré, n'importe quel composant de l'arborescence peut interagir directement avec le Store sans passer par les `props` de ses parents, grâce à deux Hooks fournis par `react-redux`.

5.1 Lire l'État Global avec `useSelector`

Le Hook **`useSelector`** permet d'extraire une donnée précise depuis l'arbre d'état du Store. De plus, il abonne automatiquement votre composant : dès que cette valeur change dans le Store, le composant se re-rend.

JavaScript

```
import { useSelector } from 'react-redux';

export default function StationsSummary() {
  // Extraction de la liste des stations depuis la tranche 'stations'
  const stationsList = useSelector(state => state.stations.list);

  const onlineCount = stationsList.filter(st => st.status === 'online').length;

  return (
    <div className="summary-banner">
      <p>Stations connectées : {onlineCount} / {stationsList.length}</p>
    </div>
  );
}
```

5.2 Modifier l'État Global avec `useDispatch`

Le Hook **`useDispatch`** retourne la fonction `dispatch` du Store. Vous l'utilisez dans vos gestionnaires d'événements (`onClick`, `onSubmit`) pour expédier les actions générées par votre Slice.

JavaScript

```
import { useSelector, useDispatch } from 'react-redux';
import { toggleStationStatus, setFilterStatus } from '../store/stationsSlice';

export default function StationControlList() {
  const dispatch = useDispatch();
  const { list, filterStatus } = useSelector(state => state.stations);

  // Filtrage dérivé selon l'état global
  const filteredStations = list.filter(st => {
    if (filterStatus === 'all') return true;
    return st.status === filterStatus;
  });

  return (
    <section>
      <div className="filters">
        <button onClick={() => dispatch(setFilterStatus('all'))}>Toutes</button>
        <button onClick={() => dispatch(setFilterStatus('online'))}>En
ligne</button>
        <button onClick={() => dispatch(setFilterStatus('offline'))}>Hors
ligne</button>
      </div>
```

```
      <ul>
        {filteredStations.map(station => (
          <li key={station.id}>
            <strong>{station.name}</strong> ({station.status})
            <button onClick={() => dispatch(toggleStationStatus(station.id))}>
              Changer le statut
            </button>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

6. Inspection et Voyage dans le Temps avec Redux DevTools

L'un des plus grands atouts de l'architecture Redux est sa traçabilité totale via l'extension de navigateur **Redux DevTools** :

1. **Journal des Actions (Action Log) :** Chaque action expédiée (`stations/toggleStationStatus`, `stations/addStation`) apparaît chronologiquement avec l'heure exacte et le contenu de son `payload`.
2. **Différence d'État (Diff) :** Pour chaque action, vous visualisez exactement quelle propriété de l'arbre JSON a été modifiée (avant / après).
3. **Voyage dans le temps (Time-Travel Debugging) :** Vous pouvez annuler ou rejouer n'importe quelle action passée d'un simple clic pour observer comment l'interface React se comporte à chaque étape historique.

7. Synthèse

1. **Pourquoi Redux :** Élimine le *Prop Drilling* et centralise l'état partagé de l'application dans un Store prévisible.
2. **Flux Unidirectionnel :** La Vue expédie (*dispatch*) une Action -> Le Reducer calcule le nouvel état -> Le Store met à jour la Vue.
3. **Redux Toolkit :** Standard moderne qui regroupe état initial, reducers immuables (via Immer) et actions au sein de `createSlice`.

4. **Hooks React-Redux :** `useSelector` pour lire et s'abonner à une donnée du Store, `useDispatch` pour déclencher une action.